



ZRC-1155: Multi-Token Standard for Conservation Assets

Antje Worring & Zach Kelling

Zoo Labs Foundation

January 2025

Abstract

We present ZRC-1155, the multi-token standard for the Zoo L2 chain. ZRC-1155 extends ERC-1155 and the Lux LRC-1155 standard (LP-2155) to support fungible, non-fungible, semi-fungible, and soulbound tokens within a single contract. The standard introduces token-level conservation categorization (species, habitat, research, impact, gaming), a structured token ID encoding scheme that embeds category and type information in the 256-bit identifier, batch impact tracking across token types, and a collection-wide auto-donation mechanism. ZRC-1155 enables gas-efficient management of conservation badges, citizen science achievements, research data bundles, donor tier memberships, and in-game wildlife items—use cases that would require hundreds of separate ERC-20 or ERC-721 contracts under conventional standards.

Keywords: multi-token, ERC-1155, batch operations, soulbound, conservation badges, gas efficiency

1 Introduction

The Zoo ecosystem generates hundreds of distinct token types: conservation badges awarded for field work, citizen science achievements, donor tier memberships, in-game items for Zoo gaming experiences, protection credits traded in DeFi markets, and research data bundles for DeSci collaborations. Deploying individual ERC-20 or ERC-721 contracts for each type is economically infeasible: each deployment costs gas, each contract requires separate approval and management, and multi-token transfers require $O(n)$ transactions.

ERC-1155 [1] solves this by supporting multiple token types within a single contract. A single `safeBatchTransferFrom` call moves 50 different token types in one transaction, reducing gas cost by up to 90% compared to 50 individual ERC-20 transfers.

ZRC-1155 extends ERC-1155 with conservation-specific features: token-level categorization, batch impact tracking, soulbound transfer restrictions for achievement badges, and a donation mechanism that operates across batch transfers.

1.1 Use Cases

1. **Conservation Badges:** Soulbound achievements for donations, field work, citizen science. Hundreds of badge types with varying supply.
2. **Protection Credits:** Fungible tokens representing verified conservation outcomes (hectares protected, species recovered). Traded in DeFi markets.
3. **Research Data Bundles:** Semi-fungible tokens representing satellite imagery, sensor data, and field observations. Identical within a dataset, unique across datasets.
4. **Donor Tiers:** Semi-fungible membership tokens (Bronze, Silver, Gold, Platinum) with per-tier benefits.
5. **GameFi Items:** In-game tools, habitats, consumables for Zoo gaming experiences (ZIP-0004).
6. **Conservation Portfolios:** Baskets of tokens representing diversified impact across species and regions.

1.2 Contributions

1. The complete IZRC1155 interface with conservation categorization and batch impact tracking (Section 3).
2. A structured token ID encoding that embeds conservation category and type in the 256-bit identifier (Section 4).
3. Soulbound token mechanics with burn-only revocation (Section 5).
4. Batch donation mechanism operating across heterogeneous token types (Section 6).
5. Gas cost analysis demonstrating 5–10× savings over individual contracts (Section 8).

2 Background

2.1 ERC-1155

ERC-1155 [1] defines a multi-token interface supporting:

- **Fungible tokens:** Supply > 1 , interchangeable within type.
- **Non-fungible tokens:** Supply = 1, unique.
- **Semi-fungible:** Limited supply, interchangeable within type but distinct across types.

Key operations: `safeTransferFrom`, `safeBatchTransferFrom`, `balanceOf`, `balanceOfBatch`. The `uri(id)` function returns a template URI per token type.

2.2 Soulbound Tokens

Vitalik Buterin proposed soulbound tokens (SBTs) [2] as non-transferable credentials. ERC-5192 [3] formalizes a minimal SBT interface. ZRC-1155 implements soulbound semantics at the token-type level within the multi-token framework.

3 Interface Specification

3.1 Enumerations

```
1  enum TokenCategory {
2      FUNGIBLE,           // Stackable, divisible
3      NON_FUNGIBLE,       // Unique, supply of 1
4      SEMI_FUNGIBLE,      // Fungible within type
5      SOULBOUND           // Non-transferable
6  }
7
8  enum ConservationCategory {
9      NONE,               // No classification
10     SPECIES,             // Tied to a species
11     HABITAT,             // Tied to a habitat
12     RESEARCH,            // Scientific data
13     IMPACT,              // Conservation proof
14     GAMING               // In-game item
15 }
```

Listing 1: Token and conservation category enums

3.2 Token Type Info

```
1 struct TokenTypeInfo {
2     TokenCategory tokenCategory;
3     ConservationCategory conservationCategory;
4     uint256 maxSupply;           // 0 = unlimited
5     uint256 currentSupply;
6     bool transferable;           // false for SOULBOUND
7     string speciesId;           // Empty if N/A
8     address conservationFund;    // address(0) = default
9     uint16 donationBasisPoints; // 0 = default
10 }
```

Listing 2: TokenTypeInfo struct

3.3 Core Interface

```
1 interface IZRC1155 is IERC1155,
2     IERC1155MetadataURI {
3
4     // Token Type Management
5     function tokenTypeInfo(uint256 id)
6         external view returns (TokenTypeInfo memory);
7     function createTokenType(
8         uint256 id,
9         TokenCategory tokenCategory,
10        ConservationCategory conservationCategory,
11        uint256 maxSupply,
12        bool transferable,
13        string calldata speciesId
14    ) external;
15
16     event TokenTypeCreated(
17         uint256 indexed id,
18         TokenCategory tokenCategory,
19         ConservationCategory conservationCategory,
20         uint256 maxSupply
21     );
22
23     // Batch Impact Tracking
24     function totalCollectionImpact()
25         external view returns (uint256);
26     function tokenTypeImpact(uint256 id)
27         external view returns (uint256);
28     function accountImpact(address account)
29         external view returns (uint256);
30
31     event ImpactRecorded(
32         uint256 indexed tokenId,
33         address indexed account,
34         uint256 creditsAdded,
35         uint256 totalCredits
```

```

36     );
37
38     // Collection-Wide Donation
39     function defaultConservationFund()
40         external view returns (address);
41     function defaultDonationBasisPoints()
42         external view returns (uint16);
43     function setDefaultConservationFund(
44         address fund
45     ) external;
46     function setDefaultDonationBasisPoints(
47         uint16 bps
48     ) external;
49
50     event CollectionDonation(
51         address indexed from,
52         uint256[] ids,
53         uint256[] amounts,
54         uint256 totalDonationValue
55     );
56
57     // Batch Minting
58     function mintBatch(
59         address to,
60         uint256[] calldata ids,
61         uint256[] calldata amounts,
62         bytes calldata data
63     ) external;
64     function mintSoulbound(
65         address to,
66         uint256 id,
67         uint256 amount,
68         bytes calldata data
69     ) external;
70 }

```

Listing 3: IZRC1155 interface

4 Token ID Encoding

ZRC-1155 uses a structured 256-bit token ID:

Token ID (uint256):

```

+-----+-----+-----+
| Category (8 bits)| Type (48 bits) | Serial (200 bits)|
+-----+-----+-----+
    Bits 255-248       Bits 247-200       Bits 199-0

```

```

1  library ZRC1155TokenId {
2      function encode(
3          ConservationCategory category,
4          uint48 typeId,
5          uint200 serial
6      ) internal pure returns (uint256) {
7          return (uint256(category) << 248)

```

Token	Category	Type	Serial
Species badge	SPECIES (1)	0x000001	0
Habitat credit	HABITAT (2)	0x000042	0
Research dataset	RESEARCH (3)	0x000100	17
Impact proof	IMPACT (4)	0x000200	0
Game sword	GAMING (5)	0x001000	99

Table 1: Example token ID encoding for various conservation asset types.

```

8           | (uint256(typeId) << 200)
9           | uint256(serial);
10      }
11
12      function category(uint256 id)
13          internal pure returns (
14              ConservationCategory) {
15          return ConservationCategory(
16              uint8(id >> 248));
17      }
18
19      function typeId(uint256 id)
20          internal pure returns (uint48) {
21          return uint48(id >> 200);
22      }
23
24      function serial(uint256 id)
25          internal pure returns (uint200) {
26          return uint200(id);
27      }
28  }
```

Listing 4: ZRC1155TokenId library

Definition 4.1 (Token ID Uniqueness). *For any two token IDs a and b , if $category(a) = category(b)$ and $typeId(a) = typeId(b)$ and $serial(a) = serial(b)$, then $a = b$. The encoding is bijective.*

The structured encoding enables efficient on-chain filtering. Indexers and subgraphs decode the conservation category directly from the token ID without additional storage reads.

5 Soulbound Mechanics

Tokens with `transferable = false` enforce:

1. `safeTransferFrom` and `safeBatchTransferFrom` MUST revert for soulbound token IDs.
2. `setApprovalForAll` grants no transfer rights for soulbound tokens.
3. Soulbound tokens CAN be burned by the holder (voluntary credential revocation).
4. Standard `TransferSingle/TransferBatch` events are emitted on mint and burn for indexer compatibility.

```

1  function _beforeTokenTransfer(
2      address operator,
3      address from,
4      address to,
5      uint256[] memory ids,
6      uint256[] memory amounts,
7      bytes memory data
8  ) internal override {
9      super._beforeTokenTransfer(
10         operator, from, to, ids, amounts, data);
11         // Allow mints (from == 0) and burns (to == 0)
12         if (from != address(0)
13             && to != address(0)) {
14             for (uint256 i = 0; i < ids.length; i++) {
15                 require(
16                     _tokenTypes[ids[i]].transferable,
17                     "ZRC1155: soulbound"
18                 );
19             }
20         }
21     }

```

Listing 5: Soulbound enforcement in `_beforeTokenTransfer`

6 Batch Donation Mechanism

When `safeBatchTransferFrom` is called with fungible token types that have `donationBasisPoints` > 0 :

1. For each token ID in the batch, compute the donation amount using the token-specific or collection-default rate.
2. Transfer donation amounts to the respective conservation fund.
3. Transfer remaining amounts to the recipient.
4. Emit a single `CollectionDonation` event with aggregated data.

Soulbound and non-fungible tokens are exempt from donation deduction.

Proposition 6.1 (Batch Donation Gas Cost). *Let n be the number of token IDs in a batch transfer, $k \leq n$ be the number with nonzero donation rate. The donation overhead is $O(k)$ additional storage writes (one per conservation fund) plus $O(1)$ for the event emission. For $k = 0$, the batch transfer has zero donation overhead.*

7 Metadata URI Schema

The `uri(id)` function returns a template URI:

`https://api.zoo.ngo/metadata/1155/{id}.json`

Operation	Individual Contracts	ZRC-1155 Batch
Transfer 1 type	65,000 gas	52,000 gas
Transfer 10 types	650,000 gas	98,000 gas
Transfer 50 types	3,250,000 gas	290,000 gas
Mint 10 types	1,500,000 gas	180,000 gas

Table 2: Gas cost comparison: individual ERC-20/721 contracts vs. ZRC-1155 batch operations.

```

1  {
2    "name": "RainforestGuardianBadge",
3    "description": "Awardedforprotecting100
4    hectaresoftropicalrainforest.",
5    "image": "ipfs://Qm.../rainforest-badge.png",
6    "properties": {
7      "tokenCategory": "SEMI_FUNGIBLE",
8      "conservationCategory": "HABITAT",
9      "transferable": false,
10     "maxSupply": 10000,
11     "speciesId": "",
12     "habitatType": "TropicalRainforest",
13     "impactCreditsPerUnit": 100,
14     "requirements": "Donate>=1000Z00to
15     rainforestconservationfund"
16   }
17 }

```

Listing 6: ZRC-1155 metadata example

8 Gas Cost Analysis

The gas savings come from two sources: (1) reduced base transaction overhead (one transaction instead of n), and (2) reduced storage reads (single contract state instead of n contract states). For batch transfers of 50 types, ZRC-1155 uses 91% less gas.

9 Security Analysis

9.1 Supply Overflow

`mintBatch` MUST check `currentSupply + amount <= maxSupply` for each capped token type and revert atomically if any type overflows.

9.2 Soulbound Bypass

Transfer restrictions are enforced in `_beforeTokenTransfer`, not in external wrappers. This prevents bypass via `delegatecall` or proxy contracts.

9.3 Batch Gas Limits

Extremely large batch operations may exceed block gas limits. Implementations SHOULD set a maximum batch size (e.g., 100 token IDs per call).

9.4 Token ID Collisions

`createTokenType` MUST revert if a token type ID already exists. The encoding scheme guarantees uniqueness when the `ZRC1155TokenId` library is used.

9.5 Reentrancy

`safeTransferFrom` calls `onERC1155Received` on the recipient. Implementations MUST use reentrancy guards.

9.6 Donation Griefing

An attacker could create many small transfers where the donation rounds to zero. Implementations SHOULD skip donation for amounts below a minimum threshold.

10 Related Work

ERC-1155 [1] defines the base multi-token standard. Enjin [4] pioneered ERC-1155 for gaming assets. ERC-5192 [3] defines minimal soulbound token interfaces; ZRC-1155 integrates soulbound mechanics within the multi-token framework. OpenSea’s Wyvern protocol [5] handles ERC-1155 marketplace listings. Thirdweb [6] provides ERC-1155 drop contracts for batch minting.

In conservation, Reef Conservation Tokens [14] use ERC-1155 for coral reef protection credits. ZRC-1155 generalizes this pattern with formal conservation categorization, structured token IDs, and soulbound badge support.

POAP (Proof of Attendance Protocol) [13] uses ERC-721 for event attendance badges; ZRC-1155’s soulbound SEMI_FUNGIBLE category achieves the same goal with significantly lower gas cost for batch issuance.

11 Conclusion

ZRC-1155 provides a gas-efficient, conservation-aware multi-token standard for the Zoo ecosystem. The structured token ID encoding enables efficient on-chain categorization without additional storage. Soulbound mechanics ensure that achievement badges represent genuine participation. The batch donation mechanism extends conservation funding to every multi-token transfer. Together, these features make ZRC-1155 the practical foundation for the hundreds of token types the Zoo conservation ecosystem requires.

Acknowledgments

This work was supported by Zoo Labs Foundation. We thank the Enjin team for pioneering the ERC-1155 standard and OpenZeppelin for the reference implementation.

References

- [1] W. Radomski et al., “EIP-1155: Multi Token Standard,” Ethereum Improvement Proposals, 2018.
- [2] V. Buterin, E. Weyl, and P. Ohlhaber, “Decentralized Society: Finding Web3’s Soul,” 2022.

- [3] T. Wilson et al., “EIP-5192: Minimal Soulbound NFTs,” Ethereum Improvement Proposals, 2022.
- [4] Enjin, “Enjin: Blockchain Game Development Platform,” Technical Documentation, 2019.
- [5] D. Finzer and A. Atallah, “OpenSea: Decentralized Marketplace for Crypto Collectibles,” Technical Documentation, 2018.
- [6] thirdweb, “ERC-1155 Drop: Batch Minting and Claiming,” Technical Documentation, 2023.
- [7] Lux Partners, “LP-2155: LRC-1155 Multi-Token Standard,” Lux Protocol Standards, 2022.
- [8] Zoo Labs Foundation, “ZIP-0004: Gaming Standards for Zoo Ecosystem,” Zoo Improvement Proposals, 2024.
- [9] Zoo Labs Foundation, “ZIP-0100: Zoo Contract Registry,” Zoo Improvement Proposals, 2024.
- [10] Zoo Labs Foundation, “ZIP-0500: ESG Principles for Conservation Impact,” Zoo Improvement Proposals, 2024.
- [11] Zoo Labs Foundation, “ZIP-0700: ZRC-20 Fungible Token Standard,” Zoo Improvement Proposals, 2025.
- [12] Zoo Labs Foundation, “ZIP-0701: ZRC-721 NFT Standard,” Zoo Improvement Proposals, 2025.
- [13] POAP, “Proof of Attendance Protocol,” <https://poap.xyz>, 2019.
- [14] Reef Conservation, “Reef Conservation Tokens: ERC-1155 for Marine Protection,” Technical Report, 2022.
- [15] OpenZeppelin, “OpenZeppelin Contracts,” <https://openzeppelin.com/contracts>, 2023.
- [16] C. Reitwießner et al., “EIP-165: Standard Interface Detection,” Ethereum Improvement Proposals, 2018.
- [17] F. Vogelsteller and V. Buterin, “EIP-20: Token Standard,” Ethereum Improvement Proposals, 2015.
- [18] W. Entriken et al., “EIP-721: Non-Fungible Token Standard,” Ethereum Improvement Proposals, 2018.
- [19] Zoo Labs Foundation, “ZIP-0015: Zoo L2 Chain Architecture,” Zoo Improvement Proposals, 2024.
- [20] Z. Leibowitz et al., “EIP-2981: NFT Royalty Standard,” Ethereum Improvement Proposals, 2020.